

CS 30700: Sprint Planning 1

BoilerSpace

Team 3: Aagam Bapna, Sheehan Dandapat, Rohan Potta, Nathan Wan, Jed Macapinlac, Bidit Acharyya

Sprint Overview

Distillation:

During this sprint, being our very first, we aim to implement the foundational infrastructure of BoilerSpace. Our goal is to establish secure user authentication, enable basic real-time room availability tracking, and support note upload and retrieval for courses. This sprint focuses on building a functional vertical slice of the application using the MERN stack, ensuring frontend-backend-database connectivity and testing coverage of such listed properties.

Scrum Master: Sheehan Dandapat

Meeting Plan: Monday/Wednesday/Friday @ 4:30 PM in WALC 3076

Risks and Challenges:

Being our very first spring, there are many challenges and risks that we are meant to run into. First, it will be important to have communication and overcome issues on that front, as it will be our first time hands-on with the project. Also, we must handle concurrent room check-ins without inconsistent room counts so that all users see the same numbers. We must also design MongoDB schemas early without requiring major refactors late. Finally, we must all respect managing workload evenly while maintaining clean Git workflow practices

Backlog (Cross out what's finished or in progress)

Functional Requirements:

Login/Profile

- ~~1. As a user, I would like to register for a BoilerSpace account so that I can save my preferences and access the full suite of features.~~
- ~~2. As a user, I would like to be able to login and manage my BoilerSpace account so that I can keep my personal data secure and up to date.~~
- ~~3. As a user, I would like to be able to verify my Purdue email by getting an OTP so that the platform remains exclusive to the campus community and prevents spam.~~
- ~~4. As a user, I would like my password to be reset if I forget it so that I can regain access to my account without creating a new one.~~
- ~~5. As a user, I would like to be able to edit my profile (display name, major, year) so that others recognize me in sessions.~~
6. As a user, I want to restrict/block another user, so that I don't see their sessions or messages.
7. As a user, I want to set my public/private profile visibility so that I control what

others see.

- ~~8. As a user I want to set my classes for this semester, so that others know what I am studying~~
9. As a user, I want to be able to friend others based on what classes they are taking so that I can build a network of study partners for my specific coursework.
10. As a user, I want to update my availability or preferred study times, so that others know when I'm typically open to studying.
11. As a user, I want to manage my notification preferences from my profile, so that I can control how and when I'm contacted.
12. As a user, I want the option to upload a profile picture so others can recognize me more easily.
13. As a user, I want to be able to delete my account so that I can remove my personal data if I leave the university or stop using the app, and we will also have 2FA.

Insights

14. As a user, I want to see my total study time for this week, so that I can track my productivity.
15. As a user, I want to receive achievements for hitting study goals so that I stay motivated to maintain a consistent study routine.
16. As a user, I want AI recommendations based off my study habits so that I can improve efficiency
17. As a student, I would like to receive notifications for when a room is under a certain capacity so that I can travel to the location knowing I will successfully find a seat.
18. As a student, I want to see a clear explanation when location access is requested, so that I understand why it is needed before granting permission.
19. As a student, I want to receive reminders for upcoming study sessions I joined, so that I don't forget to attend.
20. As a student, I want to receive notifications when new content is uploaded to courses, sessions, or events that I follow so that I stay up to date without manually checking the platform.
21. As a student, I want to temporarily mute all notifications, so that I can focus without interruptions.
22. As an admin, I want to deactivate abusive accounts so that the community stays safe, if someone gets reported too many times, we will remove them from our platform.
23. As an admin, I want to verify official campus organizations, so that students can trust event legitimacy.
24. As an admin, I want to remove inappropriate content (notes, comments, or

events), so that the platform remains respectful.

25. As a student, I want to confirm or dispute a previous user vibe report so that the live data remains accurate and up to date
26. As a student, I want to see a timestamp of last update for a room status so I can judge how reliable the information is before I start walking
27. As a student, I want to filter the map for quiet zones specifically so I can find a place for deep focus rather than group work.
28. As a student, I want to export an AI generated study guide to PDF or markdown so I can study offline or print it out
29. As a student, I want to see a simple green or red dot next to a room name so I can instantly know if it is likely open or full
- ~~30. As a student, I want to click a single button to check in to a room so I can contribute to the live data without filling out a long form~~
- ~~31. As a student, I want to see a basic list of buildings sorted alphabetically so I can find my favorite study spot quickly~~
- ~~32. As a student, I want to see how many people are currently checked into a specific building so I can gauge the overall crowd level~~
- ~~33. As a student, I want to use a search bar to find a specific building by its name or campus abbreviation~~
34. As a student, I want my dashboard and room pages to display recent buildings I've visited, key room amenities (such as whiteboards and outlets), and the last updated time of room status so that I can quickly choose a study space that fits my needs and trust the availability information.
- ~~35. As a student, I want to filter my course list by the current semester so I can focus only on the classes I am taking right now~~
36. As a student, I want to see a small icon next to a building name that indicates if it has a cafe or vending machines nearby so I can choose a study spot based on snack availability
37. As an admin, I want automatic spam detection so less moderation work is needed
38. As an admin, I want to be able to broadcast announcements so users can receive platform updates
39. As an admin, I want to see platform analytics to see which features are most popular, least-used, and make data-driven decisions to improve the platform
- ~~40. As a student looking for a seat, I want to view an interactive campus map so that I can quickly see buildings and rooms.~~
41. As a student I want directions to different study locations so that I can find the fastest route to a building I haven't visited before.
42. As a student, I want to see a list of rooms sorted by distance from my current location so that I can find the nearest available spot.

- ~~43. As a student, I want to bookmark my favorite rooms so that I can quickly check them in the future.~~
- 44. As a student I would like to find people from my class to study with so that we can help each other understand course material.
- 45. As a student, I want to search for certain classes that people are studying for so that I can join a group that is working on the same assignment/project as me.
- 46. As a student, I want to search for certain clubs and if they are hosting any study get-togethers so that I can meet people with shared interests.
- 47. As a student, I want to comment on a session's notes so that I can ask clarifying questions or provide corrections to shared content.
- 48. As a student, I want to know certain exam dates for certain classes so that I can prioritize my study sessions effectively.
- 49. As a student, I want the option of dark mode so that I can study more comfortably at night.
- 50. As a student, I want to adjust the text size, so that all the content is easier to read.
- ~~51. As a student, I want to view and upload notes from study sessions for my course so that I can both review material I may have missed and contribute to the shared knowledge base.~~
- ~~52. As a student, I want to upvote/downvote certain notes so that the most helpful and accurate resources rise to the top.~~
- 53. As a student, I want to rate and post reviews on study areas so others can find the best spots
- 54. As a student, I want to filter rooms based off amenities (whiteboards, outlets) so that I can find a workspace that fits my equipment needs.

AI Assistant

- 55. As a student, I want to ask AI assistant questions about the aggregated study notes for my course so that I can find specific explanations without reading every document.
- 56. As a student, I want the AI assistant to show citations that link back to the exact note or page it used, so that answers are traceable and verifiable.
- 57. As a student, I want the AI assistant to generate concise study guides from my notes and highlight key topics, so I can review efficiently and know what to focus on.
- 58. As a student, I want the AI to suggest practice questions without giving full solutions, so that I can test my understanding.
- 59. As a student, I want to bookmark helpful AI responses, so that I can easily return to them later.
- 60. As a student, I want the AI to help create a study plan and schedule so I can

organize my time efficiently

61. As a student, I want AI to suggest external resources like textbooks or videos to improve my understanding of certain topics

Clubs / Organizations

- ~~62. As a club organizer, I want to post events with time and location so that students can discover them easily.~~
63. As a club organizer, I want to tag events by category (callout, workshop, social, fundraiser) so that students can filter them.
64. As a club organizer, I want to create an event with a title, description, date, time, and campus location, so that students clearly understand what the event is.
65. As a club organizer, I want to manage who can create events under my club so that only authorized members can post.
66. As a club organizer, I want to attach external links (sign-up forms, Discord, social media) so that students can engage beyond the event.
67. As a club organizer, I want to see how many students plan to attend so that I can plan resources.
- ~~68. As a club organizer, I want to post announcements or updates related to events, so that attendees stay informed.~~
69. As a club member I want to RSVP to events so that the organizers know to expect me.
70. As a club organizer, I want to restrict even visibility to certain audiences (member-only or public), so that events are targeted.
- ~~71. As a club organizer, I want to create and manage a club profile, so that students can learn about the organization.~~
72. As a club organizer, I want to assign different roles to club members (e.g., admin, officer) so that responsibilities are clear.
73. As a club organizer, I want to approve or remove club members, so that organizations stay moderated.
74. As a club organizer, I want to schedule recurring events so members can plan attendance in advance
75. As a club member, I want to see files or slides from past events to stay up to date
76. As a club organizer, I want to see past attendance statistics so I can evaluate engagement

Communication

77. As a student, I want threaded discussions for session pages, so that conversations are organized. (if time allows)
78. As a student, I want to reply to specific messages within a session, so that side

conversations stay contained within the main discussion thread.

79. As a student, I want to receive notifications when someone replies to a thread I participated in, so that I can stay engaged without monitoring the entire chat.

80. As a student, I want to mute specific threads within a session, so that I can focus on conversations that matter to me.

81. As a student, I want to see a timestamp on messages in session discussions, so that conversations have context.

82. As a student, I want to search within a session's discussion threads, so that I can find specific information.

83. As a student, I want to react to messages with simple reactions, so that I can acknowledge messages without replying.

84. As a student, I want to be able to delete messages if I accidentally send something and have a message appear when I do so that others know I deleted a message

Non-Functional Requirements

Architecture and Performance

BoilerSpace will use a client server architecture based on the MERN stack (MongoDB, Express, Node.js) to support maintainability and future growth. The frontend will be a responsive web application built with React and Tailwind CSS, while the backend will provide a REST API built with Node.js and Express to manage user accounts, room availability, study sessions, file uploads, and AI-assisted queries. This separation allows system components to be developed and updated independently. The system will be designed to provide responsive performance under typical usage. Common actions such as viewing room availability, browsing study sessions, and loading the campus map should be completed within 1 second. More resource intensive actions including operations with the AI Knowledge Chatbot should be completed within 3 seconds. The application is expected to support approximately 300 concurrent users without noticeable performance degradation. Authentication and authorization will be implemented using Passport.js within our Express backend. User passwords will be salted and hashed using bcrypt (with a work factor of 10) before storage in MongoDB. To secure the REST API, we utilize the passport-jwt strategy, requiring a valid, signed JSON Web Token or JWT in the authorization header for all protected endpoints. This stateless authentication method prevents Cross-Site Request Forgery and ensures data scraping is blocked.

Crowdsourced Data Integrity (Security)

BoilerSpace relies on crowdsourced check-ins to provide live room availability and study environment indicators. Room data should be based on combined input from multiple users, while still allowing recent individual check-ins to influence results when limited data is available. The system should account for users leaving rooms by

gradually reducing the impact of older check-ins over time, ensuring that availability indicators stay aligned with real campus conditions.

Usability

The BoilerSpace interface emphasizes clear organization to help users efficiently discover study spaces, sessions, and campus events. The interactive campus map enables users to explore locations, view ability, and access relevant information with minimal effort. Study sessions, courses, and club events are structured with filtering and search options to make content easy to find and sort. The AI chatbot is designed to be easily accessible within the application, allowing users to ask questions and review course related content without disrupting their workflow. Users should also be easily able to view past notes and materials shared by other students or generated through the AI assistant. Profile features such as class selection and visibility controls are streamlined so users can manage how they are represented to others. The application will provide a consistent and usable experience across devices, screen sizes, and modern web browsers.

Hosting/Deployment

To maintain continuous delivery and scalability, the application will use a split deployment strategy. The frontend will be hosted on Vercel, which provides automatic deployments from our GitHub repo and a global CDN for fast load times. The backend will be deployed on Render. This way we can implement our crowdsourcing and messaging features, which require a long-running server process.

Current Sprint Detail

User Story 31: Basic Building and Room Retrieval

Story: As a student, I want to see a basic list of buildings sorted alphabetically so I can find my favorite study spot quickly.

Tasks (Jed)

1. **Design Building and Room database schemas — 3.0 hrs**
Define structured data models for Buildings and Rooms, including required fields and relationships necessary to support occupancy tracking.
2. **Implement automated database seeding process — 3.5 hrs**
Create a repeatable initialization script that loads a representative dataset of buildings and rooms without creating duplicates.
3. **Implement building and room retrieval APIs — 3.0 hrs**
Develop backend endpoints that return building listings and room listings in a consistent, sorted format.
4. **Testing and validation coverage — 1.5 hrs**
Add automated tests to verify schema validation and correct API responses.

Estimated total: 11.0 hrs

Acceptance Criteria

- Given the database is seeded, when I call GET/buildings, then I receive an alphabetically sorted list of buildings.
- Given a building exists, when I call GET/buildings/:id/rooms, then I receive only rooms that belong to that building.
- Given I rerun the seed script, when it completes, then buildings/rooms are not duplicated.
- Given I pass an invalid building id, when I call the endpoint, then I receive an error or not found.

User Story 30: Crowdsourced Room Check-In System

Story: As a student, I want to click a single button to check in to a room so I can contribute to the live data without filling out a long form.

Tasks (Jed)

1. **Design occupancy tracking data model — 3.0 hrs**
Create a check-in data model that tracks user presence in rooms and supports automatic expiration.
2. **Implement check-in API endpoint — 4.0 hrs**
Develop backend functionality allowing authenticated users to register an active presence in a room while preventing duplicate active check-ins.
3. **Configure automatic expiration behavior — 2.0 hrs**

Implement time-based expiration logic so that inactive check-ins are automatically removed from occupancy calculations.

4. Testing and validation — 2.0 hrs

Add tests to verify duplicate prevention and expiration behavior.

Estimated total: 11.0 hrs

Acceptance Criteria

- Given I am authenticated, when I call POST/rooms/:roomId/checkin, then a check-in record is created for my user and that room.
- Given I already have an active check-in, when I try to check in again, then the API prevents duplicates.
- Given check-ins expire after the defined window, when time passes beyond expiration, then expired check-ins no longer count as active occupancy.

User Story 32: Mock Real-Time Occupancy & Aggregation

Story: As a student, I want to see how many people are currently checked into a specific room or building so I can gauge crowd levels.

Tasks (Jed)

1. Implement room-level occupancy calculation — 4.0 hrs

Develop backend logic that calculates active occupancy for a room using only non-expired check-ins and returns structured status data.

2. Implement building-level occupancy aggregation — 3.5 hrs

Develop aggregation queries that compute total active occupancy across all rooms within a building.

3. Testing and verification of aggregation logic — 2.5 hrs

Validate that occupancy totals correctly reflect active check-ins and exclude expired entries.

Estimated total: 10.0 hrs

Acceptance Criteria

- Given multiple users are actively checked into a room, when I call GET/rooms/:roomId/status, then the returned activeCount equals the number of non-expired check-ins for that room.
- Given a check-in has expired based on the defined expiration window, when I call the room status endpoint, then that expired check-in is not included in the occupancy count.
- Given multiple rooms exist within a building and have active check-ins, when I call GET/buildings/:buildingId/occupancy, then the total occupancy equals the sum of active room check-ins within that building.
- Given no active check-ins exist for a room or building, when I call the status endpoint,

then the returned occupancy count is zero.

User Story 71: Club Profile Creation & Management

Story: As a club organizer, I want to create and manage a club profile so that students can learn about the organization.

Tasks (Nathan)

1. **Design club profile database schema — 3.0 hrs**
Define structured data model for clubs including name, description, contact info, category, and organizer ownership fields.
2. **Implement club creation and update APIs — 3.5 hrs**
Develop backend endpoints that allow authorized organizers to create and update club profiles while validating required fields.
3. **Implement club profile retrieval endpoint — 2.0 hrs**
Create API endpoint allowing users to fetch club profile information for display in the frontend.
4. **Testing and validation coverage — 1.5 hrs**
Add automated tests to verify schema validation, authorization checks, and correct API responses.

Estimated total: 10.0 hrs

Acceptance Criteria

- Given I am an authorized organizer, when I submit club information, then a new club profile is successfully created.
- Given a club profile exists, when I request its information, then I receive correct club details.
- Given I am not authorized to edit the club, when I attempt to update the profile, then the request is rejected.
- Given required fields are missing, when I submit club creation, then the API returns a validation error.
- Given a club profile is updated, when users view it, then they see the updated information.

User Story 62: Event Creation by Club Organizers

Story: As a club organizer, I want to post events with time and location so that students can discover them easily.

Tasks (Nathan)

1. **Design event database schema — 3.0 hrs**
Define event data model including title, description, date, time, location, and associated

club.

2. Implement event creation API endpoint — 3.5 hrs

Develop backend endpoint allowing authorized club organizers to create events associated with their club.

3. Implement event retrieval endpoint — 2.0 hrs

Create API route that allows users to fetch events for viewing and discovery.

4. Testing and validation coverage — 1.5 hrs

Add automated tests verifying event creation, schema validation, and correct retrieval responses.

Estimated total: 10.0 hrs

Acceptance Criteria

- Given I am an authorized organizer, when I create an event, then it is stored successfully.
- Given events exist, when users request events, then they receive event details.
- Given required event fields are missing, when creation is attempted, then the API rejects the request.
- Given a user is not authorized, when they attempt to create an event, then the request is denied.
- Given an event is created, when users browse events, then the event appears in listings.

User Story 68: Event Announcements & Updates

Story: As a club organizer, I want to post announcements or updates related to events so that attendees stay informed.

Tasks (Nathan)

1. Design announcement data model — 3.0 hrs

Define database schema for event announcements including event reference, message content, timestamp, and author information.

2. Implement announcement creation API — 3.5 hrs

Develop backend endpoint allowing authorized organizers to post announcements linked to events.

3. Implement announcement retrieval endpoint — 2.0 hrs

Create API endpoint enabling users to fetch announcements related to a specific event.

4. Testing and validation coverage — 1.5 hrs

Add tests verifying correct announcement creation, retrieval, and authorization behavior.

Estimated total: 10.0 hrs

Acceptance Criteria

- Given I am an authorized organizer, when I post an announcement, then it is linked to

the correct event.

- Given announcements exist, when users request event updates, then they receive all announcements for that event.
- Given a user is not authorized, when they attempt to post an announcement, then the request is denied.
- Given announcement content is missing, when submission is attempted, then the API returns a validation error.
- Given announcements are posted, when users view the event page, then they see the announcements in chronological order.

User Story 4: Password Reset Functionality

Story: As a user, I would like my password to be reset if I forget it so that I can regain access to my account without creating a new one.

Tasks (Rohan)

1. **Design and Implement Password Token Logic — 3.0 hrs**
Develop the backend utility functions to generate secure, time-sensitive reset tokens and hash them for database storage using the defined AuthController specifications.
2. **Implement "Forgot Password" API Endpoint — 3.5 hrs**
Create the `/auth/forgot-password` endpoint that validates the user's email, generates a reset token, and handles the logic for a "mock" email service (logging the token) for the development environment.
3. **Implement "Reset Password" API Endpoint — 2.0 hrs**
Develop the endpoint that accepts the token and the new password string, validates the token's expiration, and updates the user's password hash in the MongoDB User collection.
4. **Testing and validation coverage — 1.5 hrs**
Write unit tests to verify token expiration logic and integration tests to ensure a user cannot login with an old password after a reset.

Estimated total: 10.0 hrs

Acceptance Criteria

- Given a valid registered email, when I send a request to the forgot password endpoint, then a unique reset token is generated and associated with my account.
- Given a valid reset token and a new password, when I submit the reset form, then my password in the database is updated to the new value.
- Given an expired or invalid token, when I attempt to reset my password, then the system rejects the request with an error message.
- Given a successful password reset, when I attempt to log in with the old password, then the authentication fails.
- Given a non-existent email, when I request a password reset, then the system returns a

generic message to prevent user enumeration.

User Story 35: Course Catalog Retrieval

Story: As a student, I want to see a list of valid Purdue courses (e.g., CS 30700) so that I can select the correct ones for my profile and finding study partners.

Tasks (Rohan)

1. **Design Course Schema and Models — 2.5 hrs**
Define the Mongoose schema for the Course model as per the design doc (including fields for course code, title, and semester) and establish relationships/references to the User schema.
2. **Implement Course Data Seeding Script — 3.5 hrs**
Write a robust database initialization script to populate the MongoDB database with a standardized list of Purdue courses (CS, MA, etc.) to serve as the "single source of truth" for the application.
3. **Implement Course Retrieval API — 2.5 hrs**
Develop the `GET /courses` and `GET /courses/:id` endpoints in the ResourceController to allow the frontend to fetch the full catalog or specific course details.
4. **Testing and Schema Validation — 1.5 hrs**
Add automated tests to verify schema validation and correct API responses.

Estimated total: 10.0 hrs

Acceptance Criteria

- Given the database is empty, when the seed script is run, then the Course collection is populated with the predefined list of Purdue courses.
- Given the courses are seeded, when I call `GET /courses`, then I receive a JSON list including "CS 30700" and "MA 26100".
- Given I search for a specific course ID, when I call the API, then I receive the correct title and semester details for that course.
- Given a user attempts to create a profile, when they request the course list, they receive the data structured strictly according to the defined schema.

User Story 51: Course Note Upload and Retrieval

Story: As a student, I want to view and upload notes from study sessions for my course so that I can both review material I may have missed and contribute to the shared knowledge base.

Tasks (Rohan)

1. **Design Note Database Schema — 2.5 hrs**

Create the Mongoose schema for the Note model, ensuring it links correctly to User (uploader), Course, and StudySession as defined in the Class Level Design.

2. Implement File Upload Middleware— 3.5 hrs

Configure multer middleware in the ResourceController to handle `multipart/form-data` requests, validating file types (PDF/Images) and file sizes before passing data to the handler.

3. Implement Note CRUD API Endpoints — 3.0 hrs

Develop the `POST /courses/:id/notes` endpoint to save note metadata/file paths and the `GET /courses/:id/notes` endpoint to retrieve the list of resources for a specific class.

4. Testing and validation coverage — 1.5 hrs

Write tests to verify that invalid file types are rejected and that uploaded notes are successfully associated with the correct course ID in the database.

Estimated total: 10.5 hrs

Acceptance Criteria

- Given I have a valid PDF file, when I send a POST request to the upload endpoint, then a new Note record is created in the database.
- Given I attempt to upload a file larger than the allowed limit (e.g., 16MB), when the request is processed, then the server returns a 400 error.
- Given notes exist for "CS 30700", when I call the retrieval endpoint, then I receive a list of note objects containing the uploader's ID and the file URL.
- Given a note is uploaded, when I check the database, then the timestamp is automatically recorded.
- Given an invalid course ID, when I try to upload a note, then the operation fails with a "Course not found" error.

User Story 1: User Registration

Story: As a user, I would like to register for a BoilerSpace account so that I can save my preferences and access the full suite of features.

Tasks (Aagam)

1. Design User Schema in MongoDB — 2.5 hrs

Create the user data model with fields for email, hashed password, display name, major, year, email verification status, and timestamps. Include validation rules for required fields and email format.

2. Implement password hashing with bcrypt — 1.5 hrs

Set up bcrypt with work factor of 10 to hash passwords before storing them in the database. Ensure passwords are never stored in plain text.

3. Build POST /auth/register endpoint — 3.5 hrs

Create the registration API that accepts email, password, and basic profile info. Validate

input, check for duplicate emails, hash password, and save new user to database. Return appropriate success or error responses.

4. **Connect registration endpoint to frontend form — 2.0 hrs**

Build React registration form with input fields for email, password, confirm password, display name, major, and year. Add client side validation and connect to backend API.

5. **Testing and validation coverage — 1.5 hrs**

Write tests to verify duplicate email rejection, password hashing, successful user creation, and proper error handling for invalid inputs.

Estimated total: 11.0 hrs

Acceptance Criteria

- Given I am on the registration page, when I enter a valid Purdue email, password, display name, major, and year, then my account is created successfully and I receive a confirmation message.
- Given I try to register with an email that already exists, when I submit the form, then I receive an error message saying the email is already registered.
- Given I enter a password, when it is saved to the database, then it is stored as a bcrypt hash and not as plain text.
- Given I enter mismatched passwords in password and confirm password fields, when I try to submit, then I receive an error before the request is sent to the backend.
- Given I leave required fields empty, when I try to register, then I receive validation errors for each missing field.

User Story 2: User Login and Authentication

Story: As a user, I would like to be able to login and manage my BoilerSpace account so that I can keep my personal data secure and up to date.

Tasks (Aagam)

1. **Set up Passport.js with JWT strategy — 2.5 hrs**

Configure Passport.js authentication middleware using passport jwt strategy. Define how tokens are extracted from request headers and verified.

2. **Build POST /auth/login endpoint — 3.0 hrs**

Create login API that accepts email and password, validates credentials against database, compares hashed passwords, and returns a signed JWT token on success.

3. **Implement JWT token generation and signing — 1.5 hrs**

Set up JWT signing with a secure secret key. Include user id and email in token payload. Set appropriate expiration time for tokens.

4. **Create protected route middleware — 2.0 hrs**

Build middleware that verifies JWT tokens on protected endpoints. Extract user info from valid tokens and attach to request object. Return 401 errors for invalid or missing tokens.

5. **Testing and validation coverage — 1.5 hrs**

Write tests for successful login, failed login with wrong password, failed login with non-existent email, token verification, and protected route access.

Estimated total: 10.5 hrs

Acceptance Criteria

- Given I have a registered account, when I enter my correct email and password, then I receive a valid JWT token and am logged into the application.
- Given I enter an incorrect password, when I try to login, then I receive an error message and no token is issued.
- Given I try to access a protected endpoint without a token, when the request is made, then I receive a 401 unauthorized error.
- Given I have a valid JWT token, when I include it in the authorization header of a protected request, then the request succeeds and I can access the resource.
- Given my JWT token has expired, when I try to use it, then I receive an error and must login again.

User Story 3: Purdue Email Verification (OTP)

Story: As a user, I would like to be able to verify my Purdue email by getting an OTP so that the platform remains exclusive to the campus community and prevents spam.

Tasks (Aagam)

1. **Design OTP schema and storage — 1.5 hrs**
Create data model to store OTP codes with user email, code value, expiration timestamp, and verification status. Set OTP codes to expire after 15 minutes.
2. **Implement OTP generation logic — 1.5 hrs**
Create function to generate random 6 digit OTP codes. Store code in database with user email and expiration time.
3. **Set up email service integration — 2.5 hrs**
Integrate email sending service like Nodemailer or SendGrid to send OTP codes to Purdue email addresses. Configure SMTP settings and email templates.
4. **Build POST /auth/send-otp endpoint — 2.0 hrs**
Create API that accepts Purdue email, validates email domain is @purdue.edu, generates OTP, stores it, and sends email with the code.
5. **Build POST /auth/verify-otp endpoint — 2.0 hrs**
Create API that accepts email and OTP code, checks if code matches and is not expired, updates user verification status in database, and returns success or error.
6. **Testing and validation coverage — 1.5 hrs**
Write tests for OTP generation, expiration logic, successful verification, failed verification with wrong code, and email domain validation.

Estimated total: 11.0 hrs

Acceptance Criteria

- Given I register with a Purdue email, when registration completes, then I automatically receive an OTP code via email to that address.
- Given I receive an OTP code, when I enter the correct code within 15 minutes, then my email is marked as verified in the database.
- Given I enter an incorrect OTP code, when I try to verify, then I receive an error message and my email remains unverified.
- Given my OTP code is older than 15 minutes, when I try to use it, then I receive an expiration error and must request a new code.
- Given I try to register with a non Purdue email domain, when I submit the form, then I receive an error saying only Purdue emails are allowed.

User Story 5: User Profile Editing

Story: As a user, I would like to be able to edit my profile (display name, major, year) so that others recognize me in sessions.

Tasks (Bidit)

1. **Design User Profile Update Schema — 2.5 hrs**
Update the User schema so that fields like displayName, major, and year are editable. Include validation rules, such as required fields (e.g. displayName) and data types (String for displayName).
2. **Implement User Profile Update API — 3.5 hrs**
Develop the POST /users/:id endpoint that accepts authenticated requests, validates the input data, and securely updates the user's information in the database.
3. **Implement Current User Retrieval API — 2.5 hrs**
Develop the GET /users/:id endpoint to allow the authenticated user to fetch their current profile details for display in the edit form.
4. **Testing and Validation Coverage — 1.5 hrs**
Add automated tests to verify that only authenticated users can update their profiles and that invalid inputs are rejected.

Estimated total: 10.0 hrs

Acceptance Criteria

- Given I am logged in, when I update my display name, then the updated name is saved and reflected when I retrieve my profile.
- Given I am logged in, when I update my major, then the updated major is saved and reflected when I retrieve my profile.
- Given I am logged in, when I update my year, then the updated year is saved and reflected when I retrieve my profile.
- Given I attempt to update my profile with invalid data (e.g. a class year before the current

- one), then the API rejects the request with a validation error.
- Given I am not logged in, when I attempt to update a profile, then the request is rejected with a 401 unauthorized error.

User Story 52: User Voting

Story: As a student, I want to upvote/downvote certain notes so that the most helpful and accurate resources rise to the top.

Tasks (Bidit)

5. **Update Note Schema — 2.5 hrs**
Update the Note schema by adding a voteCount field and an array of voters to prevent multiple votes from same users.
6. **Implement Voting API — 3.5 hrs**
Develop the POST /notes/users/:id endpoint that accepts vote type (whether it is upvote or downvote). Handle logic for updating voteCount and preventing duplicate votes here.
7. **Create UI Voting Component — 2.5 hrs**
Develop upvote and downvote arrow buttons in React. Display up-to-date voting statistics for each post (e.g. net vote count, number of votes).
8. **Testing and Validation Coverage — 1.5 hrs**
Add automated tests to verify vote logic (preventing duplicate votes or changing vote type) and verify that voteCount changes correctly.

Estimated total: 10.0 hrs

Acceptance Criteria

- Given I have not voted on a note, if I upvote it, then the net vote count should increase by 1.
- Given I have already voted on a note, if I try to make the same vote again, there should be no change (no duplicate votes).
- Given I upvote a note and then click downvote later, the net vote count should decrease by 2.
- Given I refresh the page after voting, then my vote should still be visible after the page reloads.
- Given I am not logged in, when I attempt to make a vote , then the request is rejected with a 401 unauthorized error.

User Story 8: User Class Selection

Story: As a user I want to set my classes for this semester, so that others know what I am studying.

Tasks (Bidit)

9. Update User and Course Schema — 2.5 hrs

Update the User schema to include a Mongoose reference array for selected courses using course IDs.

10. Implement Update User Courses API — 3.5 hrs

Develop the POST /users/:id/courses endpoint that allows users to set or update their courses for the semester. Validate the course ID input using the Course collection.

11. Create UI Course Selection Component — 2.5 hrs

Create a dropdown menu in React that allows for multiple selections. Pull the courses using the GET /courses endpoint created in User Story 8. Allow users to save their selected courses to their profile

12. Testing and Validation Coverage — 1.5 hrs

Add automated tests to verify only valid course IDs can be saved and invalid IDs are rejected, and also tests to confirm that updates to course selections are saved.

Estimated total: 10.0 hrs

Acceptance Criteria

- Given there is a valid course list, when I select my courses and save them, then my profile stores those course IDs.
- Given I provide an invalid course ID, when I try to save it, then my request is rejected and I am shown an error.
- Given I revisit my profile after making course selections, then my selected courses should be displayed.
- Given I updated my course selections, when I save the changes, the old course list on my profile should be replaced with the updated list.
- Given I am not logged in, when I attempt to select courses, then the request is rejected with a 401 unauthorized error.

User Story 33: Map Search Bar

Story: As a student, I want to use a search bar to find a specific building by its name or campus abbreviation

Tasks (Sheehan)

1. Build Search Bar UI Component - 3.5 hrs

Create reusable React search bar component with input, search button, and results dropdown.

2. Implement Search Logic and API Integration - 5.0 hrs

Fetch building list from [GET/buildings](#) and implement matching by full name and common abbreviations (e.g., "WALC", "EE"). Add highlighting of matched substrings in the results for the client.

3. Testing and Validation Coverage - 1.5 hrs

Implement tests for empty-state UI, loading state, and API-failure messaging. Add unit

tests for filtering logic and integration tests for the component behavior.

Estimated total: 10.0 hrs

Acceptance Criteria

- Given the building list is loaded, when the user types the full building name then the matching building appears in the results.
- Given common campus abbreviations exist in the dataset, when the user types an abbreviation then the correct building matching that abbreviation appears.
- Given the query matches no building, when the user searches, then the UI displays a “No buildings found” message
- Given the search field is cleared, when the input becomes empty, then the full (alphabetical) building list is restored.
- Given the backend request fails, when the client attempts to load buildings, then a user-friendly error message is shown and the component does not crash.

User Story 43: Bookmark Favorite Rooms

Story: As a student, I want to bookmark my favorite rooms so that I can quickly check them in the future.

Tasks (Sheehan)

1. Design Bookmark Schema and Backend Routes - 4.5 hrs

Create MongoDB schema to store bookmarked rooms (userId + roomId reference). Implement **POST /users/bookmarks/:roomId** (add bookmark), **DELETE /users/bookmarks/:roomId** (remove bookmark), **GET /users/bookmarks** (retrieve bookmarked rooms).

2. Build Frontend Bookmark UI - 4 hrs

Add bookmark toggle button to room cards. Create a “My Bookmarked Rooms” section in the dashboard displaying saved rooms.

3. Testing and Validation Coverage - 1.5 hrs

Write backend unit tests for bookmark creation/removal and frontend tests for toggle behavior.

Estimated total: 10.0 hrs

Acceptance Criteria

- Given a logged-in user, when they click the bookmark icon on a room, then that room is saved to their bookmarks list.
- Given a bookmarked room, when the user clicks the bookmark icon again, then the room is removed from their bookmarks list.
- Given a user bookmarks a room, when they log out and log back in, then the

bookmarked room still appears in their saved list.

- Given a room is already bookmarked, when the user attempts to bookmark it again, then no duplicate entry is created.
- Given the user opens the “My Bookmarked Rooms” section, when data loads, then all bookmarked rooms are displayed with accurate occupancy data.
- Given the backend fails, when attempting to bookmark/unbookmark, then a user-friendly error message is displayed and the UI does not break.

User Story 40: Interactive Campus Map

Story: As a student looking for a seat, I want to view an interactive campus map so that I can quickly see buildings and rooms.

Tasks (Sheehan)

1. Build Map UI Component (React + MapBoxAPI) - 4.5 hrs

Add a responsive Map view component using React that supports pan, zoom, and marker layers. Use MapBoxAPI for Purdue campus information. Add basic UI controls: zoom buttons, a toggle for buildings/rooms layers, and a legend for status dots.

2. Populate Markers & Room Popups - 4.5 hrs

Fetch building and room data from `GET /buildings` and `GET /buildings/:id/rooms` and render building markers and room clusters. Ensure selecting a building triggers map zoom/fit and invokes the appropriate callback for other UI components (e.g., side panel or room list).

3. Testing and Validation Coverage - 1.5 hrs

Write component-level tests (React Testing Library) for rendering, marker interaction, and behavior when API returns no data.

Estimated total: 10.5 hrs

Acceptance Criteria

- Given the user opens the Map page, when data loads, then the map is centered on the campus and markers for buildings appear.
- Given building markers are visible, when the user clicks a building marker, then a popup or side panel displays building name and important information.
- Given a building is selected from the list, when the user chooses “View on map”, then the map zooms/fits to that building and opens its marker popup.
- Given focus is on the map area, when the user uses keyboard navigation (tab/arrow), then markers and controls are reachable and popups are readable.
- Given a mobile device viewport, when the user opens the map, then map controls and popups are usable (touch-friendly) and the layout does not overflow the screen.

Summary of Assignments

- **Aagam:**
 - User Story 1: User Registration
 - User Story 2: User Login and Authentication
 - User Story 3: Purdue Email Verification (OTP)
- **Bidit:**
 - User Story 8: User Class Selection
 - User Story: 52: User Voting
 - User Story: 5: User Profile Editing
- **Jed:**
 - User Story 31: Basic Building and Room Retrieval
 - User Story 32: Crowdsourced Room Check-In System
 - User Story 30: Mock Real-Time Occupancy & Aggregation
- **Nathan:**
 - User Story 71: Club Profile Creation & Management
 - User Story 62: Event Creation by Club Organizers
 - User Story 68: Event Announcements & Updates
- **Rohan:**
 - User Story 4: Password Reset Functionality
 - User Story 35: Course Catalog Retrieval
 - User Story 51: Course Note Upload and Retrieval
- **Sheehan:**
 - User Story 33: Map Search Bar
 - User Story 40: Interactive Campus Map
 - User Story 43: Bookmark Favorite Rooms